

Python If ... Else

Python Conditions and If statements

Python supports the usual logical conditions from mathematics:

- Equals: `a == b`
- Not Equals: `a != b`
- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
- Greater than or equal to: `a >= b`

```
#In this example we use two variables, a and b,  
#which are used as part of the if statement to  
#test whether b is greater than a. As a is 33,  
#and b is 200, we know that 200 is greater than 33,  
#and so we print to screen that "b is greater than a".  
  
a = 33  
b = 200  
if b > a:  
    print("b is greater than a")  
  
b is greater than a
```

These conditions can be used in several ways, most commonly in "if statements" and loops.

An "if statement" is written by using the `if` keyword.

Indentation

- Python relies on indentation (whitespace at the beginning of a line) to define scope in the code. Other programming languages often use curly-brackets for this purpose.

```
#If statement, without indentation (will raise an error):  
a = 33  
b = 200  
if b > a:  
print("b is greater than a") # you will get an error  
  
Input In [42]  
    print("b is greater than a") # you will get an error  
    ^  
IndentationError: expected an indented block
```

Elif

- The **elif** keyword is python's way of saying "if the previous conditions were not true, then try this condition".

```
a = 33
b = 33
if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
```

a and b are equal

In this example **a** is equal to **b**, so the first condition is not true, but the **elif** condition is true, so we print to screen that "a and b are equal".

Else

- The **else** keyword catches anything which isn't caught by the preceding conditions.

```
a = 200
b = 33
if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
else:
    print("a is greater than b")
```

a is greater than b

- In this example **a** is greater than **b**, so the first condition is not true, also the **elif** condition is not true, so we go to the **else** condition and print to screen that "a is greater than b". You can also have an else without the elif:

```
a = 200
b = 33
if b > a:
    print("b is greater than a")
else:
    print("b is not greater than a")
```

b is not greater than a

Shorthand If

- If you have only one statement to execute, one for if, and one for else, you can put it all on the same line:
- This technique is known as **Ternary Operators** or **Conditional Expressions**.

```
#One line if else statement, with 3 conditions:  
a = 330  
b = 330  
print("A") if a > b else print("=") if a == b else print("B")  
=  

```

```
#One line if else statement:  
a = 2  
b = 330  
print("A") if a > b else print("B")  
B  

```

And

- The **and** keyword is a logical operator, and is used to combine conditional statements:

```
#Test if a is greater than b, AND if c is greater than a:  
a = 200  
b = 33  
c = 500  
if a > b and c > a:  
    print("Both conditions are True")
```

Both conditions are True

Or

- The **or** keyword is a logical operator, and is used to combine conditional statements:

```
#Test if a is greater than b, OR if a is greater than c:  
a = 200  
b = 33  
c = 500  
if a > b or a > c:  
    print("At least one of the conditions is True")
```

At least one of the conditions is True

Nested If

- You can have if statements inside if statements, this is called *nested* if statements.

```
x = 41

if x > 10:
    print("Above ten,")
    if x > 20:
        print("and also above 20!")
    else:
        print("but not above 20.")
```

```
Above ten,
and also above 20!
```

The pass Statement

if statements cannot be empty, but if you for some reason have an if statement with no content, put in the pass statement to avoid getting an error.

```
a = 33  
b = 200  
  
if b > a:  
    pass
```

Exercise

Print "Hello World" if a is greater than b

```
a = 50
```

```
b = 10
```

```
__ a __ b __
```

```
    print("Hello World")
```

Repetition

again, and again, and again...
(Sets and Loops)

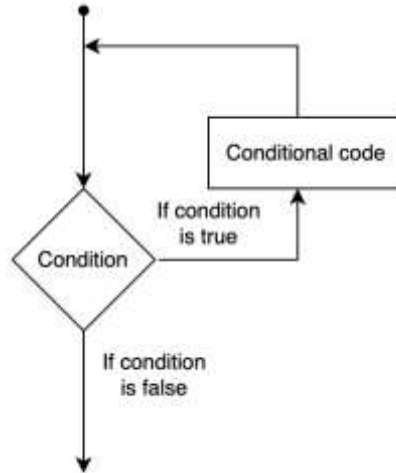
Python Loops

Loops

- In iteration control structures, a statement or block is executed until the program reaches a certain state, or operations have been applied to every element of a collection. This is usually expressed with keywords such as while, repeat, for, or do..until

Computer algorithms need to be precise

- A loop statement allows us to execute a statement or group of statements multiple times. Please see below flow chart:



Say Hello 3 times

```
print("Hello, World!")  
print("Hello, World!")  
print("Hello, World!")
```

Repetition

- Wouldn't it be great if you could just tell the computer to do something multiple times.
- Ideally, you'd say something like:

do this 3 times:

```
print("Hello, World!")
```

- You can do something like this, it's called repetition, or a loop

Types of loop:-

- While
- For

While loop

- A while loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition. The while loop can be thought of as a repeating if statement.
- With the **while** loop, we can execute a set of statements as long as a condition is true.

```
In [1]: i = 1  
        while i < 6:  
            print(i)  
            i += 1
```

```
1  
2  
3  
4  
5
```

- The **while** loop requires relevant variables to be ready, in this example, we need to define an indexing variable, **i**, which we set to 1.

The break Statement

- With the **break** statement, we can stop the loop even if the while condition is true:
- Exit the loop when **i** is 3:

```
In [2]: i = 1
        while i < 6:
            print(i)
            if i == 3:
                break
            i += 1
```

1
2
3

The continue Statement

- With the **continue** statement, we can stop the current iteration, and continue with the next:

Continue to the next iteration if i is 3:

```
In [3]: i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

1
2
4
5
6

The else Statement

- With the **else** statement, we can run a block of code once when the condition no longer is true:

Print a message once the condition is false:

```
In [4]: i = 1
        while i < 6:
            print(i)
            i += 1
        else:
            print("i is no longer less than 6")
```

1
2
3
4
5
i is no longer less than 6

For Loop

- A **for** loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).
- This is less like the **for** keyword in other programming languages and works more like an iterator method as found in other object-orientated programming languages.
- With the **for** loop, we can execute a set of statements, once for each item in a list, tuple, set etc.
- Print each fruit in a fruit list:

```
In [5]: fruits = ["apple", "banana", "cherry"]  
        for x in fruits:  
            print(x)
```

```
apple  
banana  
cherry
```

The **for** loop does not require an indexing variable to set beforehand.

- Looping Through a String: Even strings are iterable objects, they contain a sequence of characters:
- Loop through the letters in the word "banana":

```
In [6]: for x in "banana":  
        print(x)
```

b
a
n
a
n
a

The break Statement

- With the **break** statement, we can stop the loop before it has looped through all the items:
- Exit the loop when x is "banana":

```
In [7]: fruits = ["apple", "banana", "cherry"]  
        for x in fruits:  
            print(x)  
            if x == "banana":  
                break
```

```
apple  
banana
```

Exit the loop when x is "banana", but this time the break comes before the print:

```
In [8]: fruits = ["apple", "banana", "cherry"]  
        for x in fruits:  
            if x == "banana":  
                break  
            print(x)
```

apple

The continue Statement

- With the **continue** statement we can stop the current iteration of the loop, and continue with the next
- Do not print banana:

```
In [9]: fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    if x == "banana":  
        continue  
    print(x)
```

```
apple  
cherry
```

The range() Function

- To loop through a set of code a specified number of times, we can use the `range()` function,
- The `range()` function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.
- Using the `range()` function:
- Note that `range(6)` is not the values of 0 to 6, but the values 0 to 5.
- The `range()` function defaults to 0 as a starting value, however it is possible to specify the starting value by adding a parameter: `range(2, 6)`, which means values from 2 to 6 (but not including 6):

```
In [10]: for x in range(6):  
         print(x)
```

```
0  
1  
2  
3  
4  
5
```

Else in For Loop

The **else** keyword in a for loop specifies a block of code to be executed when the loop is finished

Print all numbers from 0 to 5, and print a message when the loop has ended:

```
In [14]: for x in range(6):  
         print(x)  
         else:  
             print("Finally finished!")
```

```
0  
1  
2  
3  
4  
5  
Finally finished!
```

Example

Break the loop when x is 3, and see what happens with the else block:

```
In [15]: for x in range(6):  
         if x == 3: break  
         print(x)  
     else:  
         print("Finally finished!")
```

0

1

2

Nested Loops

- A nested loop is a loop inside a loop.
- The "inner loop" will be executed one time for each iteration of the "outer loop":
- Print each adjective for every fruit:

```
In [16]: adj = ["red", "big", "tasty"]  
        fruits = ["apple", "banana", "cherry"]
```

```
        for x in adj:  
            for y in fruits:  
                print(x, y)
```

```
red apple  
red banana  
red cherry  
big apple  
big banana  
big cherry  
tasty apple  
tasty banana  
tasty cherry
```

The pass Statement

for loops cannot be empty, but if you for some reason have a for loop with no content, put in the **pass** statement to avoid getting an error.

```
In [17]: for x in [0, 1, 2]:  
         pass
```

Test Yourself With Exercises

Exercise: Loop through the items in the list of fruits.

```
fruits = ["apple", "banana", "cherry"]  
x = fruits  
print(x)
```

Show Answer